



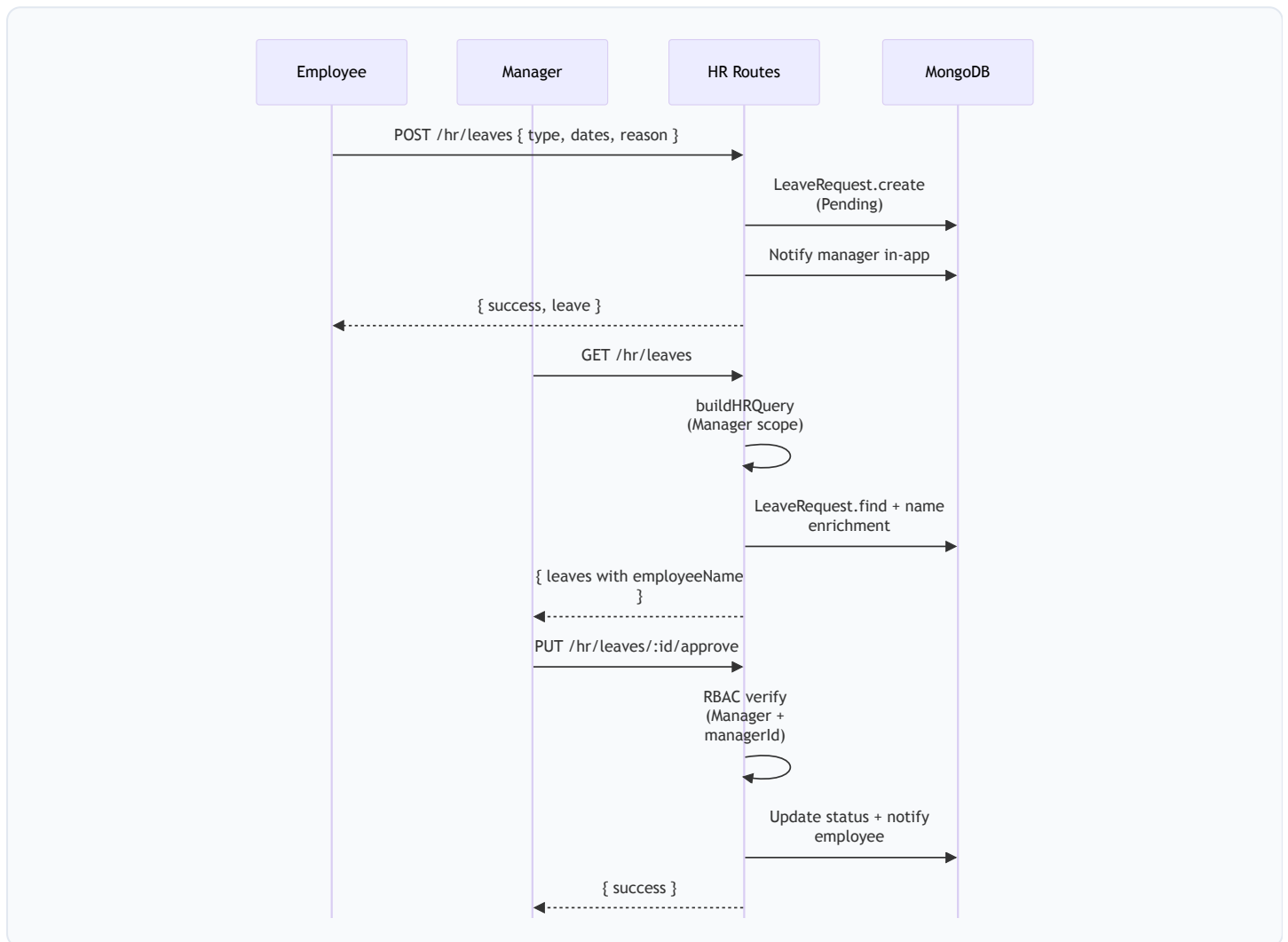
Corporate (HR/Enterprise) Architecture

HR operations, hierarchy, support platform, and deployment topology

AceTrack Architecture Documentation • Generated 12 June 2026

1. HR Module — Leave Management

The HR module uses role-based query scoping (buildHRQuery) to ensure proper data isolation: Admin sees all, Manager sees own + direct reports, Employee sees only own records.



Detailed Flow Breakdown

1 Apply for Leave

Employee submits a leave request with type (Earned/Sick/Casual), start/end dates, and reason. The server creates a `LeaveRequest` document with status: "Pending" and auto-notifies the employee's manager via in-app notification.

```
POST /api/v1/hr/leaves { type, startDate, endDate, reason }
```

2 Manager Notification

The server looks up the employee's `managerId` from their `Player` document, then pushes an in-app notification to the manager: "[Employee Name] applied for [type] leave from [date]."

3 Manager Views Team Leaves

buildHRQuery scopes the query based on role. For managers, it fetches direct reports via `Player.find({ managerId: self })`, then queries leaves for all those userIDs. Results include name enrichment (v2.6.446).

```
GET /api/v1/hr/leaves
```

4 RBAC Approval Guard

When approving/rejecting, the server verifies: (a) `req.user.supportLevel === "Manager"`, and (b) the leave's employee has `managerId` matching the requesting manager. This prevents managers from approving leaves for employees outside their team.

5 Approve Leave

Manager approves with an optional comment. The server updates the leave status and pushes a notification to the employee: "Your [type] leave was approved."

```
PUT /api/v1/hr/leaves/:id/approve { managerComment }
```

6 Reject Leave

Same RBAC verification as approval. Manager provides a rejection reason, employee is notified with the comment.

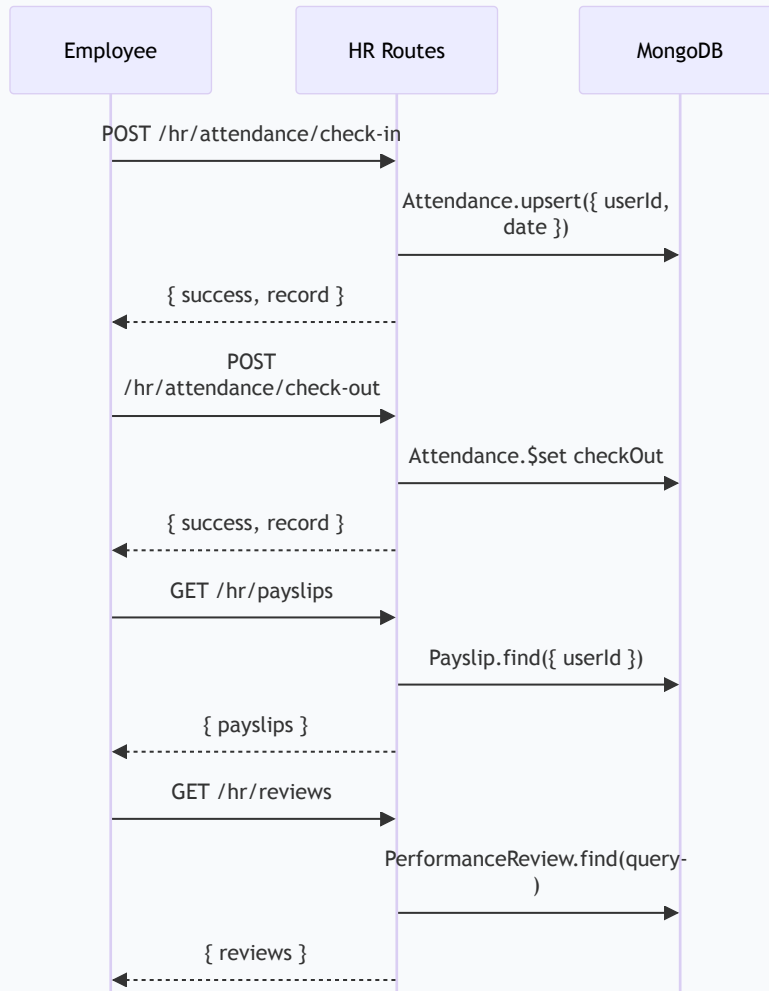
```
PUT /api/v1/hr/leaves/:id/reject { managerComment }
```

API Endpoints

ENDPOINT	METHOD	SCOPING	PURPOSE
/hr/leaves	GET	Admin/Manager/Self	List leave requests
/hr/leaves	POST	Self	Apply for leave
/hr/leaves/:id/approve	PUT	Admin/Manager	Approve leave
/hr/leaves/:id/reject	PUT	Admin/Manager	Reject leave

2. HR Module — Attendance, Payslips & Reviews

Employee self-service for daily attendance tracking, payslip viewing, performance reviews, and document management.



Detailed Flow Breakdown

1 Daily Check-In

Employee clocks in. The server uses today's date (YYYY-MM-DD) as a unique key with `findOneAndUpdate + upsert` to create or update the attendance record.

`POST /api/v1/hr/attendance/check-in`

2 Daily Check-Out

Employee clocks out. The server updates the same date's record with the check-out timestamp.

`POST /api/v1/hr/attendance/check-out`

3 View Attendance History

Returns the last 30 attendance records, scoped by `buildHRQuery`. Managers see their team's records.

`GET /api/v1/hr/attendance`

4 View Payslips

Employee views uploaded payslips sorted by upload date (newest first). Scoped by role.

`GET /api/v1/hr/payslips`

5 Performance Reviews

Employee views their performance review history. Managers can see reviews for their direct reports.

`GET /api/v1/hr/reviews`

6 HR Documents

Employee accesses uploaded HR documents (contracts, certificates, etc.) sorted by upload date.

GET /api/v1/hr/documents

7 Company Policies

All employees can view company policies, which are stored as OrgSetting entries with key prefix "policy_".

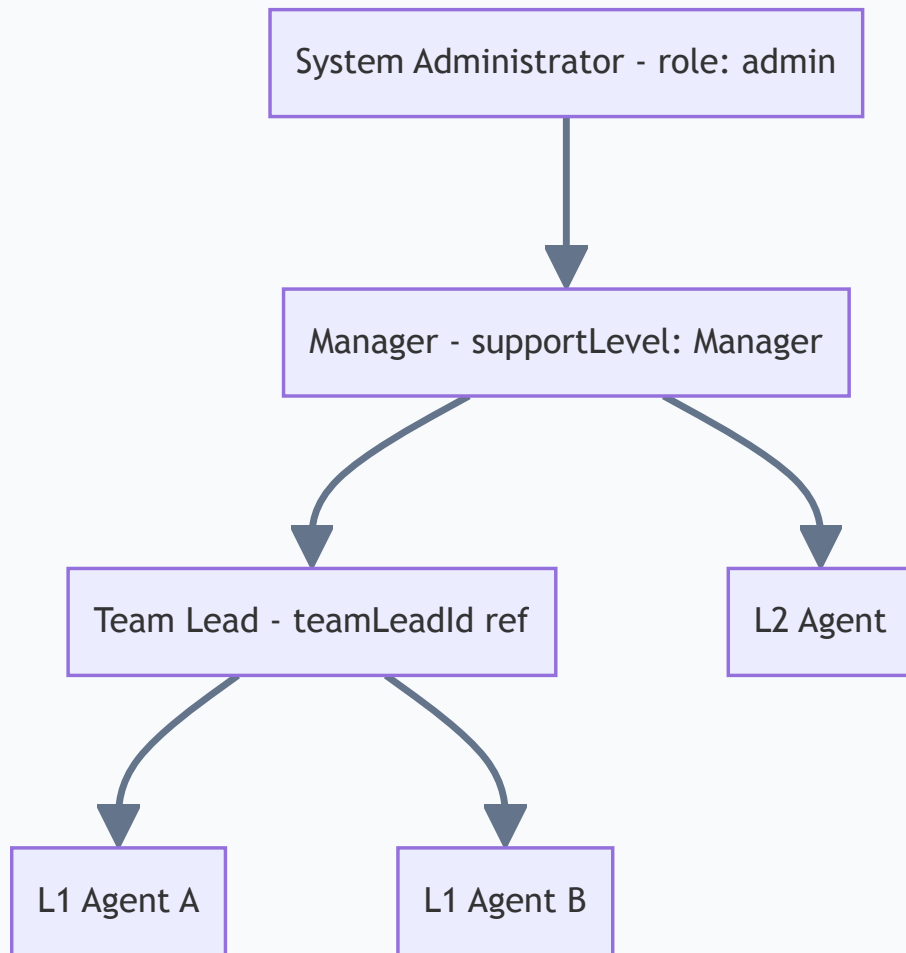
GET /api/v1/hr/policies

API Endpoints

ENDPOINT	METHOD	SCOPING	PURPOSE
/hr/attendance/check-in	POST	Self	Clock in
/hr/attendance/check-out	POST	Self	Clock out
/hr/attendance	GET	Admin/Manager/Self	View records
/hr/payslips	GET	Admin/Manager/Self	View payslips
/hr/reviews	GET	Admin/Manager/Self	View reviews
/hr/documents	GET	Admin/Manager/Self	View documents
/hr/policies	GET	All	View policies

3. Organizational Hierarchy

The reporting structure uses managerId and teamLeadId references on Player documents to establish a hierarchical chain that controls HR data visibility.



Detailed Flow Breakdown

- 1 System Administrator**
Top-level role with unrestricted access to all data, settings, and operations. Can manage staff hierarchy, view all HR records, and configure org settings.
- 2 Manager (supportLevel: Manager)**
Mid-level role with visibility into direct reports' HR data (leaves, attendance, reviews). Can approve/reject leave requests. Queries use buildHRQuery to scope data.
- 3 Team Lead**
Organizational reference via teamLeadId on Player documents. Team leads may have L1/L2 agents reporting to them. The teamLeadId is set by admin via the hierarchy endpoint.
- 4 L2 Agent**
Higher-level support agent with expanded ticket handling capabilities. Sees only their own HR records.
- 5 L1 Agent**
Entry-level support agent. Handles basic tickets and sees only their own HR records. Can be promoted to L2 or Manager via /support/manage-user.
- 6 Ex-Employee Detection**
The team directory automatically detects ex-employees by checking supportStatus === "terminated" or supportLevel === "EX-EMPLOYEE". These users are shown with "Ex-Employee" designation and isLive: false.

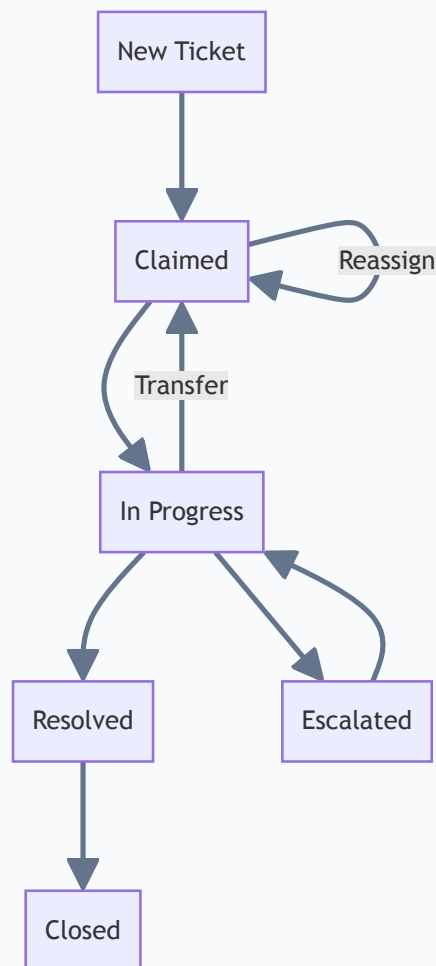
7 Setting Hierarchy

Admin updates the reporting chain via POST /admin-core/team-directory/:id/hierarchy with managerId and teamLeadId. Each change is logged in AuditLog.

```
POST /admin-core/team-directory/:id/hierarchy { managerId, teamLeadId }
```

4. Support Ticket Lifecycle

The support operations platform manages the complete ticket lifecycle from creation to resolution, with AI-powered summaries and analytics.



Detailed Flow Breakdown

1 New Ticket

Player submits a ticket with subject, description, and category. A SupportTicket document is created with status "open" and no assignee.

```
POST /api/v1/support/save-ticket { subject, description, category }
```

2 Claim

An agent claims the ticket, setting themselves as the assignee. WebSocket broadcasts the update for real-time dashboard refreshes.

```
POST /api/v1/support/claim-ticket { ticketId }
```

3 In Progress

Agent responds to the player (public reply) or adds internal notes. Messages are appended to the ticket's thread.

```
POST /api/v1/support/reply-ticket { ticketId, message, isInternal }
```

4 Escalate

Agent or admin escalates the ticket's priority or support level (L1 → L2 → Manager) for complex issues.

```
POST /api/v1/support/escalate-ticket { ticketId, escalateToLevel }
```

5 Reassign / Transfer

Ticket can be reassigned to a different agent or bulk-transferred during offboarding.

```
POST /support/reassign-ticket or /support/transfer-tickets
```

6 AI Summary

Generate an AI-powered summary of the entire ticket thread using Groq/Cerebras LLM for quick context understanding.

```
POST /api/v1/support/ai-summary { ticketId }
```

7 Resolve & Rate

Agent resolves the ticket. Player can rate the resolution quality (1-5 stars with optional feedback).

```
POST /support/update-ticket-status + /support/rate-ticket
```

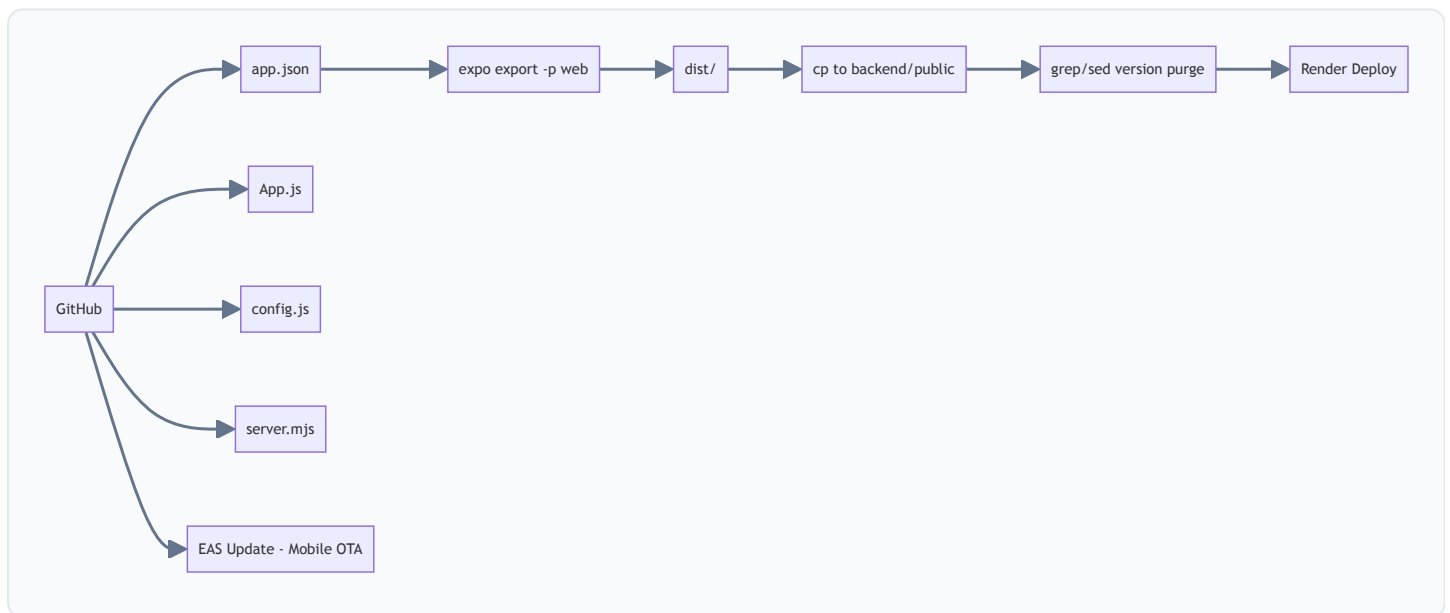
8 Analytics

Dashboard view with: open/closed counts, average resolution time, tickets per agent, SLA compliance, and export capability.

```
GET /api/v1/support/analytics
```

5. Deployment & Version Sync

The deployment architecture requires strict synchronization of version constants across 4 files and a complete web rebuild pipeline for every frontend change.



Detailed Flow Breakdown

1 Version Constants (4 Files)

Every deployment MUST synchronize APP_VERSION across: app.json (binary), App.js (frontend), config.js (initial state), and server.mjs (backend /api/status). Failure to sync server.mjs causes clients to report "Up to Date" incorrectly.

2 Web Export

Frontend changes require `npx expo export -p web` to generate the production bundle. The output goes to `dist/` folder.

```
npx expo export -p web
```

3 Deploy to backend/public

The dist/ output replaces backend/public/ which serves the web admin portal. This is the only way frontend changes become visible on the live web app.

```
rm -rf backend/public && cp -R dist backend/public
```

4 Version String Purge

A grep/sed command replaces all old version references in minified bundles. CRITICAL: Dots in version strings MUST be escaped (e.g., 2\.\6\625) to prevent sed from treating them as regex wildcards and corrupting CSS hex codes.

```
grep -irl "OLD" backend/public | xargs sed -i 's/OLD/NEW/g'
```

5 Render Deploy (Backend + Web)

Git push triggers automatic deployment on Render. The Express server serves both the API and the static web bundle from backend/public/.

6 EAS Update (Mobile OTA)

Mobile app updates are pushed via Expo Application Services. Running eas update on production, preview, and main branches delivers over-the-air updates to all mobile clients.

7 Security Middleware Stack

Every request passes through: CORS → Helmet (CSP, HSTS) → Rate Limiter → Cookie Parser → Body Parser (10MB) → Mongo Sanitize → CSP Nonce → API Key Guard → Auth Guard → CSRF Guard (admin/support routes).